INSPIRE AI

STAND-UP

Standup Meeting



© 2026 Inspire | Confidential

Share one event and choose one next action

Objective · what happened	Describe one event or result from the previous training day.
Reflective · how you felt	Name a feeling. Share only what you choose to share.
Interpretive · what it means	Explain the learning you take from that event.
Decisional · what next	State one action you will take into the rest of the day.

© 2026 Inspire | Confidential

REVIEW

Review CI Results and Their Limits



© 2026 Inspire | Confidential

Present the CI run you produced and what it does not prove

Keep the submission fixed	Every member presents their own graded submission or their part of a graded group submission. Do not revise the work or change the score.
Open your Actions run	Show the test results for your submitted commit. If blocked, show the last result or explain where the run stopped.
Comment on the evidence	Divide the 20 minutes by the group size. Ask where the log shows the tests ran and what risk remains after the checks.

© 2026 Inspire | Confidential

INSPIRE AI

LECTURE

Software Engineering Enters a New Paradigm Shift



© 2026 Inspire | Confidential

Process and collaboration kept improving, but AI changed who produces the code

Process and collaboration

Optimises how people write code

1936–1957

Turing machine

ENIAC

FORTRAN

1968

Software engineering named

Waterfall described

1987

CMM framework

CMMI 1.0

2001

Agile Manifesto

Scrum, CI adopted

2009

First DevOpsDays

Docker, Kubernetes

Productive force

Changes who writes the code

2021

GitHub Copilot preview

ChatGPT released

2025

Coding agents ship widely

Specification-driven work takes shape

© 2026 Inspire | Confidential

AI becoming the producer is where the difference in scale comes from

	PROCESS OPTIMISATION: WATERFALL TO DEVOPS	AI PARADIGM: AI CODING ONWARD
What changes	Process and collaboration	The productive force itself
Improvement curve	Gradual work on the same curve	A new curve, rebuilt in steps
Output ceiling	Set by how much people can produce	Set by model capability and engineering guardrails

Read the third row as the consequence: adding people no longer raises the ceiling. Model capability and engineering guardrails do.

© 2026 Inspire | Confidential

What do you first think of when you hear AI-native development?

Vote before we look at the four readings. Keep your reason; we come back to it at the end of the session.

1. **It writes faster for me**
   Completion, generation, bug fixing
2. **It finishes whole tasks**
   Task breakdown, execution, verification
3. **Work is split differently**
   Requirements, design, coding, testing
4. **Product and organisation change**
   Delivery form, team shape, roles

© 2026 Inspire | Confidential

AI-native development is not the first reading but the last three happening in turn

READING	WHAT ACTUALLY CHANGES
It writes faster for me	Tool upgrade. AI sits inside the process that already exists.
It finishes whole tasks	Closed task. The person defines the goal and the acceptance.
Work is split differently	Rewritten process. Handoffs and roles change with it.
Product and organisation change	Native capability. The business form changes.

Only the first reading is a tool upgrade. Judge a transition by closed tasks, a rewritten process and changed roles, not by how many AI tools are in use.

© 2026 Inspire | Confidential

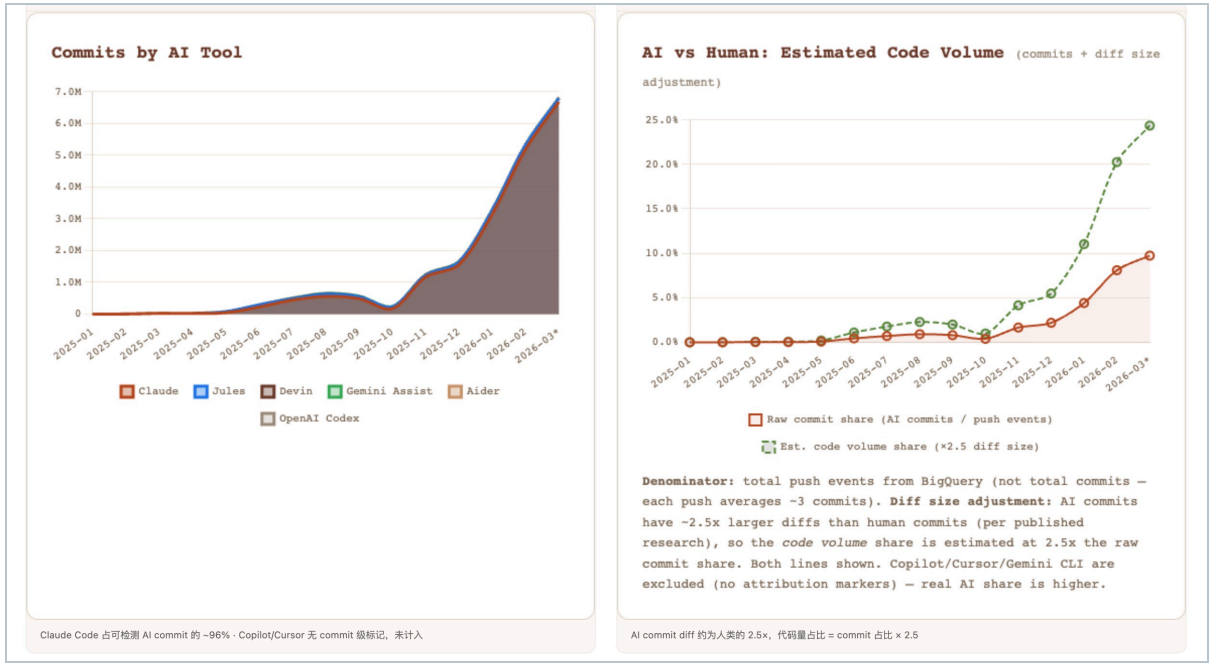
The share of code AI contributes keeps rising

VENDOR	CORE PRODUCT	AI CODE SHARE, PUBLISHED	ENGINEER ADOPTION	ROLE SHIFT
Tencent	CodeBuddy, as plugin, IDE and CLI	50% of new code in 2025, most code planned for 2026	Over 90% of engineers use it	Engineer becomes architect and AI reviewer
Alibaba	Tongyi Lingma, then Qoder CN	About 30%, October 2025	Five million Qoder users worldwide	Generation rate dropped as a KPI; effective code and defect rate kept
ByteDance	MarsCode, then TRAE	Sizeable share; internal contribution six times higher in one year	Over 80% of internal engineers use it	Engineer becomes task commander; watch for misleading metrics

Published figures and public reporting, 2025 to 2026. Scopes differ: new code, commit volume and effective code volume are not comparable.

© 2026 Inspire | Confidential

AI is moving from local completion to batch code production



Commits by AI tool, and AI against human code volume

Source: Bot Commits, Edgardo Munoz, March 2026 edition, data to 2026-03.

Left: commit share, from public data

The commit share of the main AI coding tools (Claude Code, Cursor, Jules, Gemini Code Assist) climbed fast through 2025. It shows how often AI now takes part in the work.

Right: code volume, by diff size

An AI change makes a bigger diff than a human change, so the code volume AI contributes runs above its commit share. That gap is AI moving from completion into batch production.

© 2026 Inspire | Confidential

AI usage moves from prompt to harness; tools from completion to autonomy

Once code generation is solved, the bottleneck moves to requirement understanding, environment build, test verification and release.

STAGE 1 · 2022–2024

Prompt engineering

Make the request well · 20–40%

AI USE

Role setting, few-shot examples, chain of thought

TOOL FORM

Completion and chat in the IDE; Devin names the agent

JUDGMENT

An accelerator in the editor; the person closes the task

STAGE 2 · 2024–2025

Context engineering

Bring the material · 40–70%

AI USE

Retrieval, long-term memory, context compression

TOOL FORM

MCP standardises tools; Claude Code drives act and verify

JUDGMENT

A runnable change becomes the unit of delivery

STAGE 3 · 2026 ONWARD

Harness engineering

Build the system · 80%+

AI USE

Tool routing, saved state, self-verification, observability

TOOL FORM

Managed runtimes, sandboxes and agent identity

JUDGMENT

Autonomy at scale; competition moves to runtime

© 2026 Inspire | Confidential

Delivery is accelerating, but gains sit in code writing; review, compliance and integration are the new bottleneck

63%
report revenue growth from faster delivery

91%
report longer review waits

86%
report more senior-engineer rework

77%
report higher technical debt

Where the time actually goes: one recorded change took about one hour to code and about three weeks to reach production.

All four figures come from one survey of 200 US technology decision makers, New Relic 2026, and public industry research. They describe other organisations, not this class.

© 2026 Inspire | Confidential

Will AI amplify each practice into a gain or into a risk?

Each of these four practices happened in your own work on Days 1 to 10.
Vote, then give the one reason that decided it.

1	**Unit tests and clear interface contracts** The core module can be checked and replayed after every change	GAIN	RISK
2	**Requirements only in chat history** No acceptance standard is written anywhere	GAIN	RISK
3	**Generation rate is the only metric** Delivery time and effective output are not tracked	GAIN	RISK
4	**Shared conventions and review gates** Risky changes have to pass a gate before they land	GAIN	RISK

© 2026 Inspire | Confidential

AI multiplies what is already there

PRACTICE	VERDICT	REASON THAT DECIDED IT
Unit tests and interface contracts	Gain	AI changes can be verified and replayed, so trying costs less.
Requirements only in chat	Risk	Without an acceptance standard, faster generation means faster rework.
Generation rate as the only metric	Risk	The process metric rises while the delivery cycle stays flat. The gap is hidden.
Conventions and review gates	Gain	A gate turns team agreement into a check that runs on every change.

AI does not create good practice or bad practice on its own. It multiplies what is already there. Fix the practice first, then talk about speed.

© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

LECTURE

From Agile to Specification-Driven: The Logic Underneath the Shift



© 2026 Inspire | Confidential

Each methodology was an answer to a contradiction of its time

METHODOLOGY	CONTRADICTION IT ANSWERED	WHAT IT OPTIMISED
Waterfall	Large projects lost control: hardware was expensive and software kept growing	Fixed phases, with the whole requirement defined first
Agile	Requirements kept changing while people wrote code slowly	Short cycles, continuous feedback, delivery in increments

© 2026 Inspire | Confidential

Which single assumption does Agile rest on?

Sixty seconds with the person next to you. Do not answer with a practice; answer with the assumption that makes the practice worth doing.

Take one answer from three pairs before moving on. More than one of these is a real premise of Agile. Only one of them is the premise AI has just broken.

1. **Requirements keep changing**
   So the process must absorb change instead of preventing it
2. **Code is the expensive part**
   So every practice exists to shorten the coding cycle
3. **People work better in short loops**
   So cadence matters more than the size of the batch

© 2026 Inspire | Confidential

AI breaks exactly that assumption

What Agile assumed	Coding is the bottleneck, so shorten the cycle and deliver in increments.
What AI changed	Code arrives in minutes. The expensive parts are now saying what is wanted and proving it is right.
What moves	The bottleneck leaves the middle of the chain and goes to its two ends: definition and verification.

Specification-driven work does not replace Agile. It moves the same empirical loop from managing human execution to managing the trustworthiness of an executor that never gets tired.

© 2026 Inspire | Confidential

Three uncertainties now stack on the same change

UNCERTAINTY	HOW IT FAILS	WHERE IT SHOWED UP IN THIS COURSE
Execution	Same input, different output. The wrong result looks reasonable.	A green test that asserted the wrong rule
Context	Right locally, inconsistent globally. The area stays unknown.	A form wired to a client, with a conflicting update
Intent	No tacit understanding; the agent reads only the words.	An unclear change nobody could judge

© 2026 Inspire | Confidential

Each uncertainty names what a specification must contain

UNCERTAINTY	FAILURE IT PRODUCES	WHAT THE SPECIFICATION MUST CONTAIN
Execution	Confident wrong output	Decidable expected results, not a description of the implementation
Context	Locally correct, globally inconsistent	Existing behavior and the affected scope, written out
Intent	Precise but wrong rules	Intent as a checkable contract, with unconfirmed rules left as questions

You cannot write first and align later.

© 2026 Inspire | Confidential

The core proposition: get trustworthy results from an executor you cannot trust

	TRADITIONAL ENGINEERING	SPECIFICATION-DRIVEN WORK
Executor assumption	A person with judgement, common sense and self-correction	High execution, low judgement: an amplifier of whatever it is given
What the method manages	People, tasks and progress	Specifications and verification
Where quality comes from	Skilled people and review at the end	Precise definition and gates along the way

Software is not a pile of code. It is a specification that keeps evolving, and code is one snapshot of it.

© 2026 Inspire | Confidential

Specification work pays off only when the work is formalizable and verifiable

Both high: specification work pays for itself. Either low: the cost of *stating* the requirement overtakes the value of the change.

Worth it	Enterprise apps, platform and API services, financial and compliance systems, data pipelines, products that change often.
Think twice	Zero-to-one exploration, creative work, one-off short projects, purely subjective requirements, teams new to AI.

One sentence settles it: use it when stating the requirement is worth more than writing the code.

© 2026 Inspire | Confidential

From code-centric to specification-centric is a change of worldview

DIMENSION	CODE-CENTRIC	SPECIFICATION-CENTRIC
Core asset	The code base	The living specification
First citizen	Working software	A precise specification
Bottleneck	Implementation	Definition and verification
Role of people	Writing code	Defining and verifying
Quality	Review at the end	Gates along the way
Rhythm	Time-box driven	Change driven
Measured	Human velocity	Specification quality and trust
Main risk	Changing requirements and too few people	Specification drift and verification gaps

© 2026 Inspire | Confidential

The scarce capability moves from implementation to definition

The scarce capability moves	From implementation to definition.
The human role moves	From writing code to defining and verifying.
So tomorrow starts here	Get the intent out of a person and into a form the specification can accept, before anyone writes a line.

© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Why Specifications Produce Better Results Than Vibing It



© 2026 Inspire | Confidential

What You Have

A retrospective on one tech meetup. The whole event exists in one person's head.

1. 60 seats, 112 registered, 95 turned up
2. The projector has only VGA, the speaker has USB-C, and finding an adapter took 25 minutes
3. The air conditioning could not handle 95 people; 35 degrees inside
4. Speaker 2 thought they had 30 minutes, had 20, and ran 15 minutes over
5. Speaker 4 cancelled by text that morning; there was no backup
6. 60 meals ordered against an expected 80; 95 people came
7. 8 vegetarians, and no vegetarian option
8. No water on site; the dispenser broke the day before
9. The sign-up form had no cap, no waitlist and no full message
10. The time said only 2:00 PM, with no time zone

© 2026 Inspire | Confidential

Part 2 · Your Tasks (steps 1-3)

① Task 1 · Bet first 1 min

- From the ten, write down the three you think the vague version will hit.
- Write them down and do not change them. You may only run once they are written.

② Task 2 · Run the vague version 3 min

```
Plan a tech community meetup for about 80 people with 4 speakers.
```

Hard rule: when the AI asks a question, answer only `Use your best judgement.` Add one fact and the exercise is dead.

③ Task 3 · Label every line 5 min

Fact	The venue has a projector	You can go and check it
Decision	The projector must support USB-C	Already decided for you
Action	Confirm the venue AV equipment	Pushes the problem later

An action item is not a defence. It only reschedules the same hole.

© 2026 Inspire | Confidential

Part 2 · Your Tasks (steps 4-6)

④ **Task 4 · Compare** 2 min

- How many of the ten did it hit, and are they the three you bet on?
- Count your tags: how many decisions, how many actions?

⑤ **Task 5 · Write three elements** 4 min

- Not three good ones. Three that together block the most failures.
- Then report one number: these three block N of the ten.

⑥ **Task 6 · Close with three questions** 5 min

1. Of the ten, how many are the plan inventing a fact that does not exist?
2. How many are the model not seeing enough, not enough context?
3. The rest: what caused them?

Answer in order, one line each, no skipping.

© 2026 Inspire | Confidential

Part 3 · Reflections

1. How many of the ten did the vague version hit? Are they the three you bet on?
2. What is your ratio of decisions to actions? What does that number tell you?
3. Of the ten, which did the AI invent, which did it fail to see, and which are neither?
4. Your three elements blocked how many? What did you give up to raise that number?
5. In the work you are holding now, where has someone already filled a blank for you?

Documents solve visibility. Decisions solve disagreement. A spec is finished when nobody has to guess, not when it is written.

© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

INSPIRE AI

LECTURE

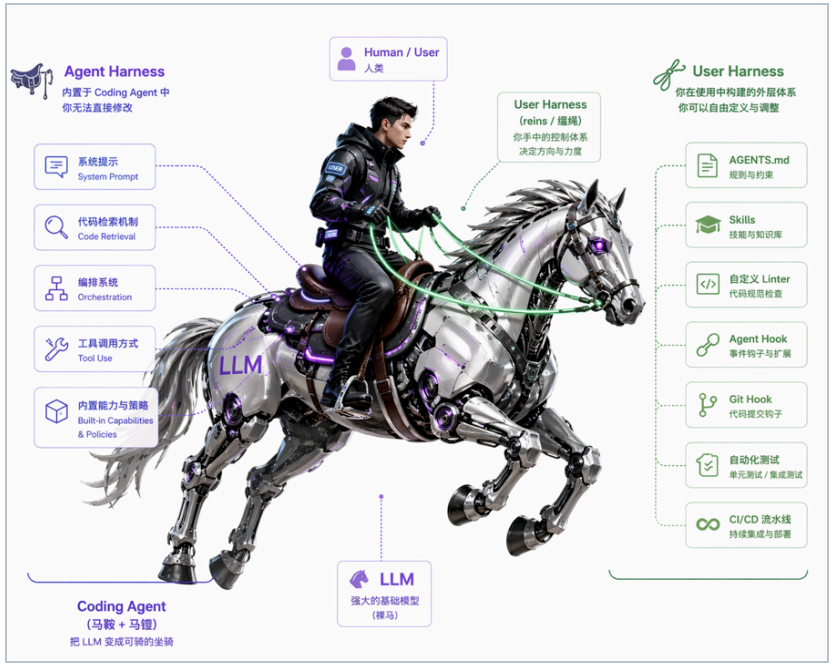
Harness engineering for coding agent users



© 2026 Inspire | Confidential

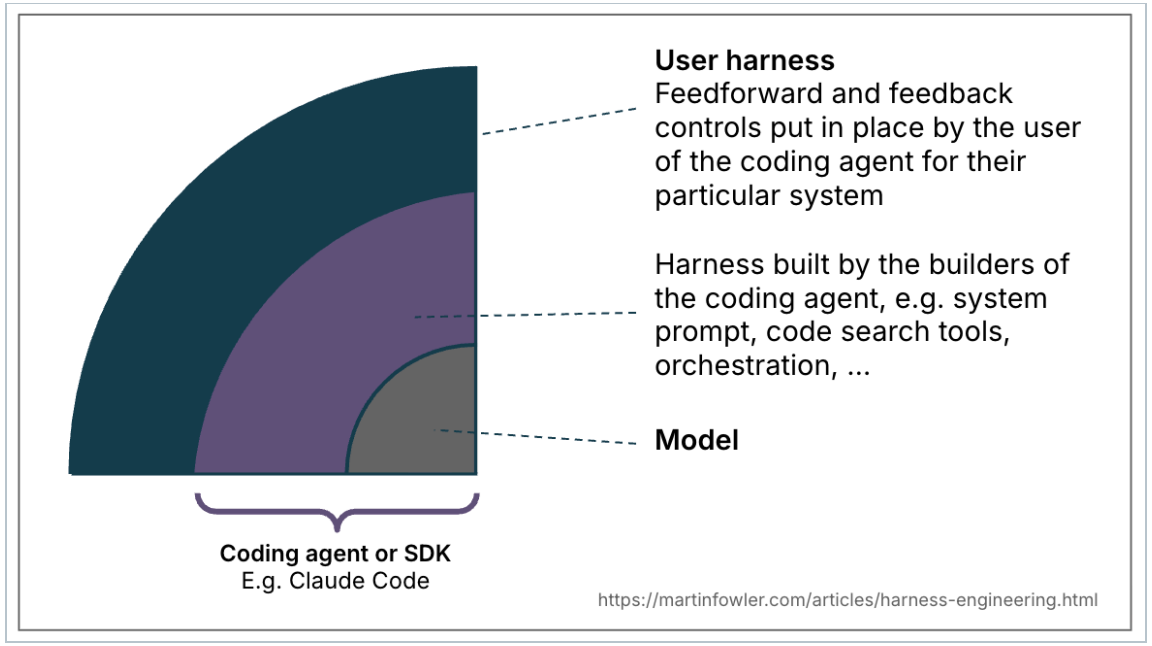
Harness engineering for coding agent users

A specification guides the coding agent through the ring you put around it.



Agent Harness is the saddle; User Harness is the reins

Diagram: Birgitta Böckeler, Harness engineering, martinfowler.com



Three nested harness layers

Diagram: Birgitta Böckeler, Harness engineering, martinfowler.com

© 2026 Inspire | Confidential

User Harness is a cybernetic governor

Feedforward plus feedback steers the codebase toward the desired state.

	COMPUTATIONAL · CPU · MILLISECONDS TO SECONDS	INFERENTIAL · GPU OR NPU · SLOWER, COSTLIER, NOT DETERMINISTIC
Guides · feedforward	LSP · CLI · bootstrap script · codemod · project template	AGENTS.md · Skills · Architecture.md · principles · CfR · rules · reference docs · how-to · feature spec
Sensors · feedback	ESLint · type checks · unit tests · ArchUnit · coverage · dependency-cruiser · pre-commit · mutation testing · logs · browser	AI review agent · architecture review skill · agent self-review loop

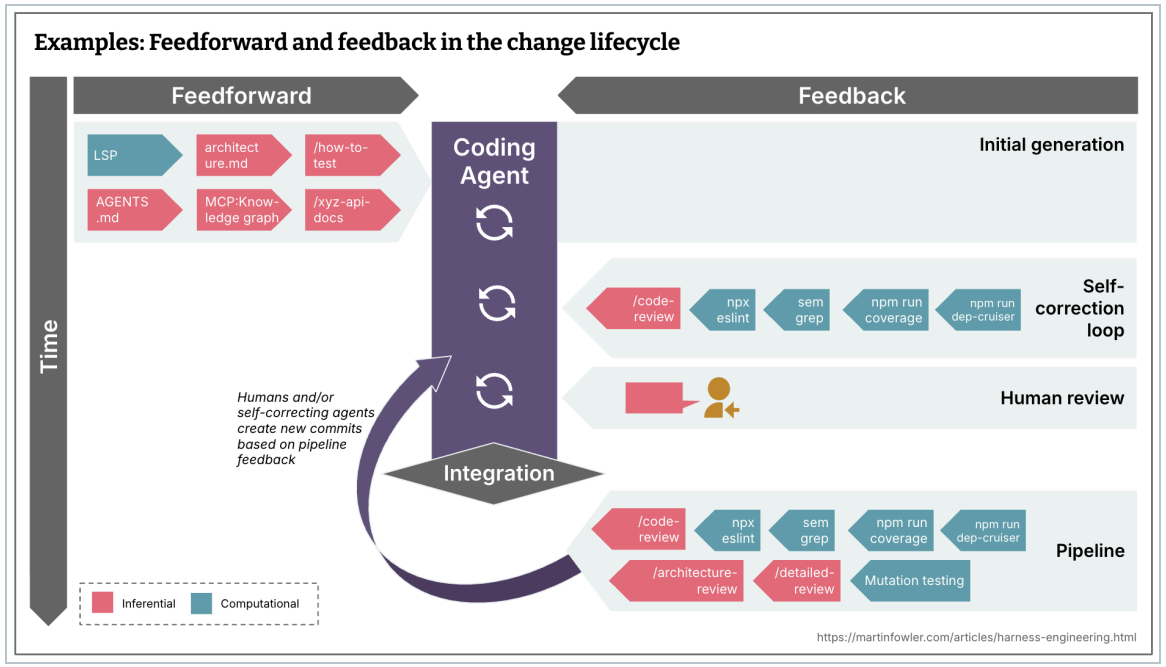
> Guides alone go unchecked; sensors alone pay twice for the same error.

One guide can span both cells: a bootstrap Skill is the words, its script is the computation.

© 2026 Inspire | Confidential

Feedforward and feedback in the change lifecycle

Fast checks next to the agent; expensive ones after integration.



Feedforward and feedback in the change lifecycle

Diagram: Birgitta Böckeler, Harness engineering, martinfowler.com

Before integration

Fast checks run in the self-correction loop. Human review is the last sensor here.

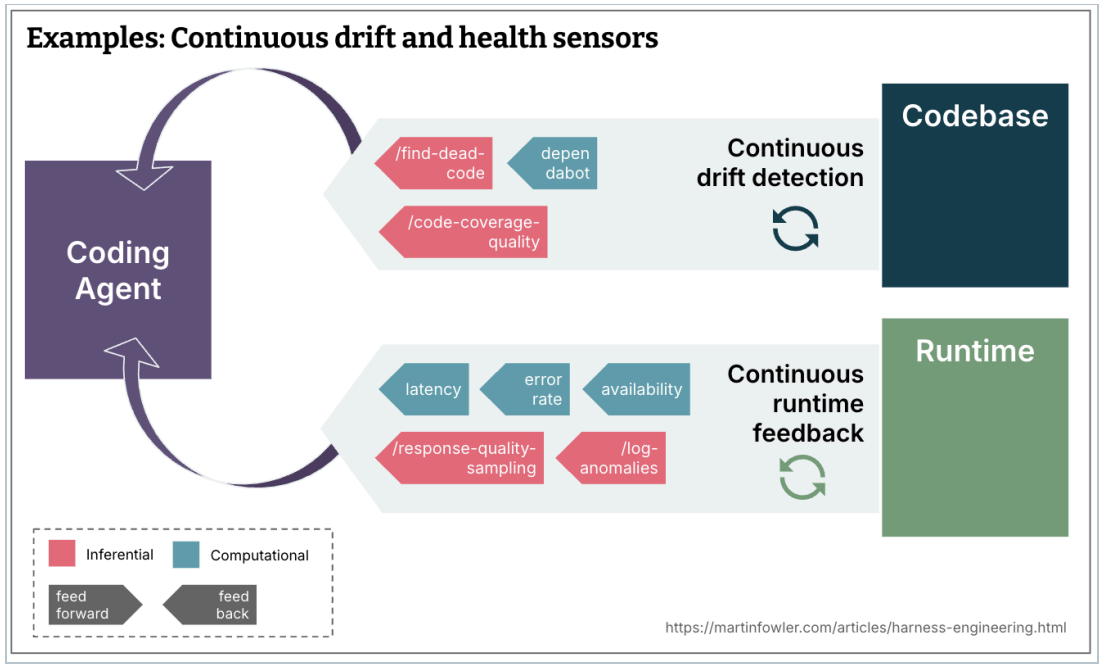
After integration

The pipeline reruns every fast check, adds the expensive ones, and returns the fix as a new commit.

© 2026 Inspire | Confidential

Continuous drift and health sensors

Degradation that no single commit caused, and that no per-commit sensor ever catches.



Continuous drift and health sensors

Diagram: Birgitta Böckeler, Harness engineering, martinfowler.com

Codebase drift

Dead code, expiring dependencies, slipping coverage. OpenAI scans and ships fixes as PRs.

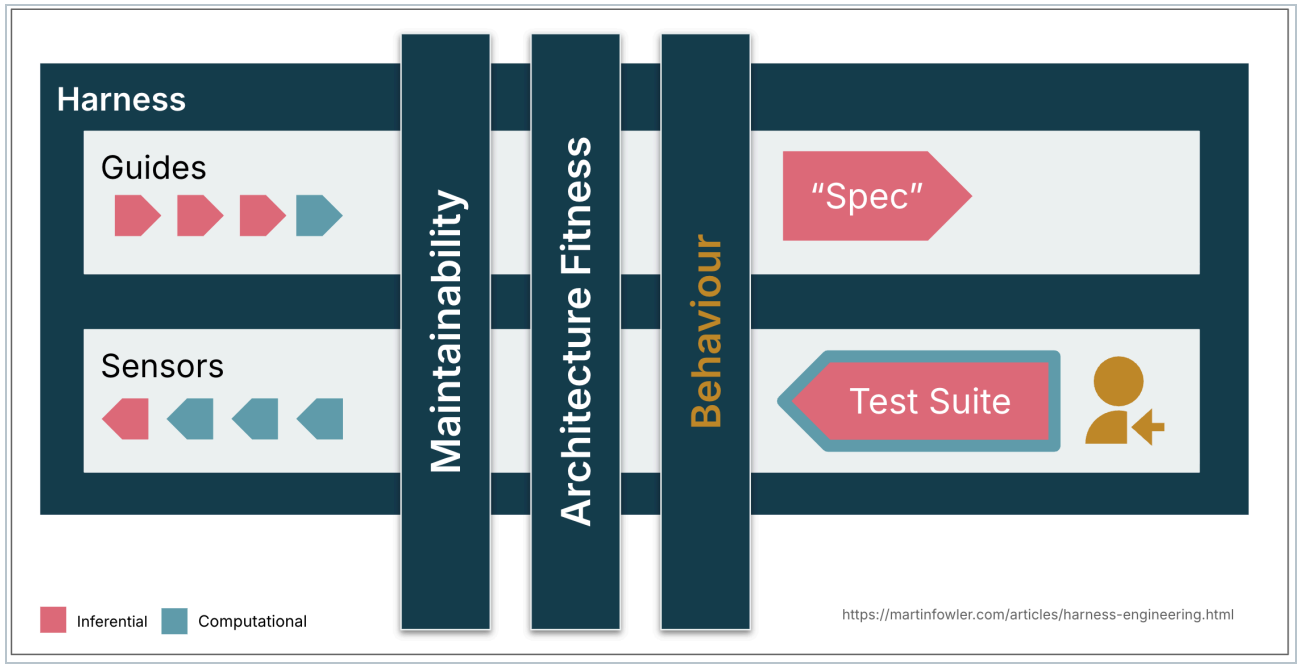
Runtime feedback

A throwaway observability stack per worktree, queried with LogQL and PromQL.

© 2026 Inspire | Confidential

Governance categories in the desired state

Guides and sensors run across; maintainability, architecture fitness and behaviour run down.



Governance categories in the desired state

Diagram: Birgitta Böckeler, Harness engineering, martinfowler.com

Maintainability

The most tooled column. Misdiagnosis and unrequested features still have no sensor.

Architecture fitness

Layering and dependency direction, pinned early and enforced mechanically.

Behaviour

The hardest column: Spec feeds forward, the test suite senses, people still review.

© 2026 Inspire | Confidential

The role of the human

When the same error appears a second time, the harness is what changes.

You hold the reins	You tune the harness; you do not walk the horse through every step.
What you bring in	Internalised conventions, a felt sense of complexity, alignment, and accountability.
What an agent lacks	No social accountability, and no way to know which conventions carry weight.

An error fixed once is code. That is what makes implicit experience explicit and external.

© 2026 Inspire | Confidential

Summary

A specification is one guide among many, and behaviour is its home column.

One side is never enough	The rule is never tested, or the same error is paid for twice.
Fast next to the agent	Expensive behind integration; two loops sit outside the lifecycle.
Twice means harness	Fix the code once. Then change the guide or the sensor.

Lead with a guide, check with a sensor, rewrite the reins the second time.

© 2026 Inspire | Confidential

BREAK

Lunch



© 2026 Inspire | Confidential

INSPIRE AI

LECTURE

What a Spec Is (and Is Not)



© 2026 Inspire | Confidential

What a Spec Is (and Is Not)

Spec first, and what the spec writes down is the behavior the outside can see.

Spec first	Written first, then handed to the agent for this task.
What a spec is	Structured, behavior-oriented, written in natural language.
What stays out	Same outside behavior, different implementation.

Spec First

Written before the code

External behavior

The only thing it constrains

© 2026 Inspire | Confidential

Spec First, beside Test First

TDD writes the test first. SDD writes the spec first. Two pairs, not one four-step pipeline.

TDD · Test Driven Development	SDD · Spec Driven Development
The discipline is *Test First*: write the test, then the implementation that passes it.	The discipline is *Spec First*: write the spec, then use it in this AI-assisted development.
The test settles first what counts as correct.	The spec settles first what this task is meant to do.

TDD pairs with SDD. Test First pairs with Spec First. This page only builds the analogy; the three ways to keep a spec are in the next page's diagram.

© 2026 Inspire | Confidential

Three levels, each keeping the spec longer

All three start from spec-first, and not all of them keep the spec afterwards.

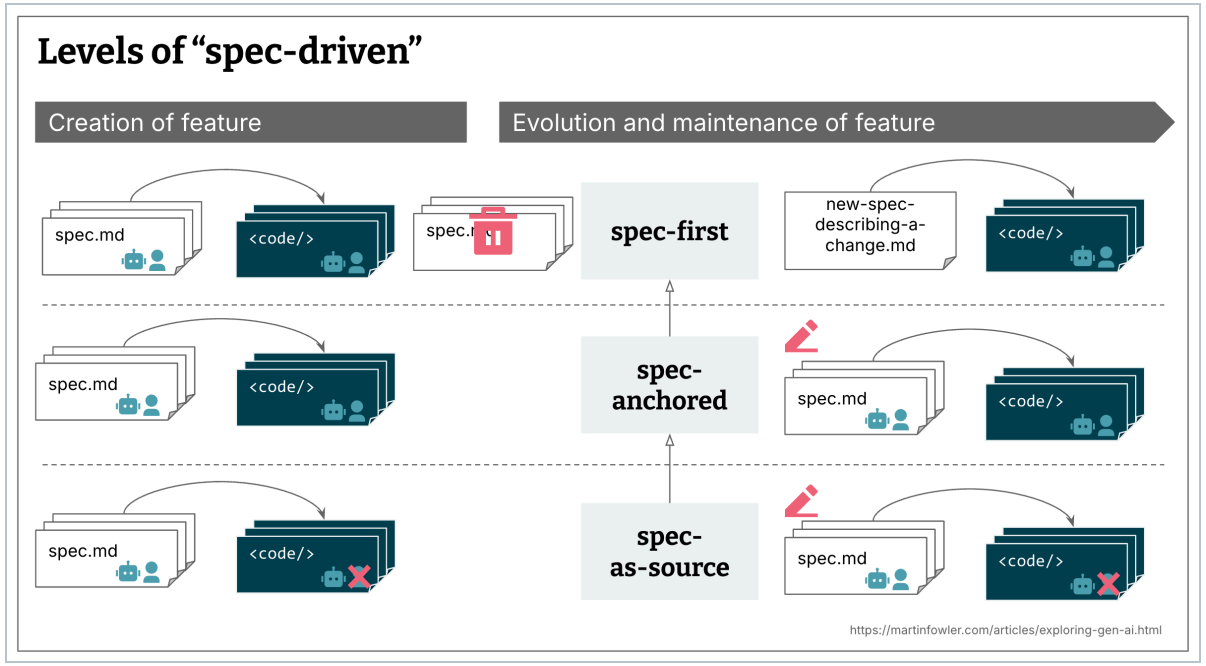


Diagram: Birgitta Böckeler, martinfowler.com

Spec-first

A well thought-out spec is written first, then used in the AI-assisted workflow for the task at hand.

Spec-anchored

The spec is kept after the task is complete, to continue using it for evolution and maintenance of that feature.

Spec-as-source

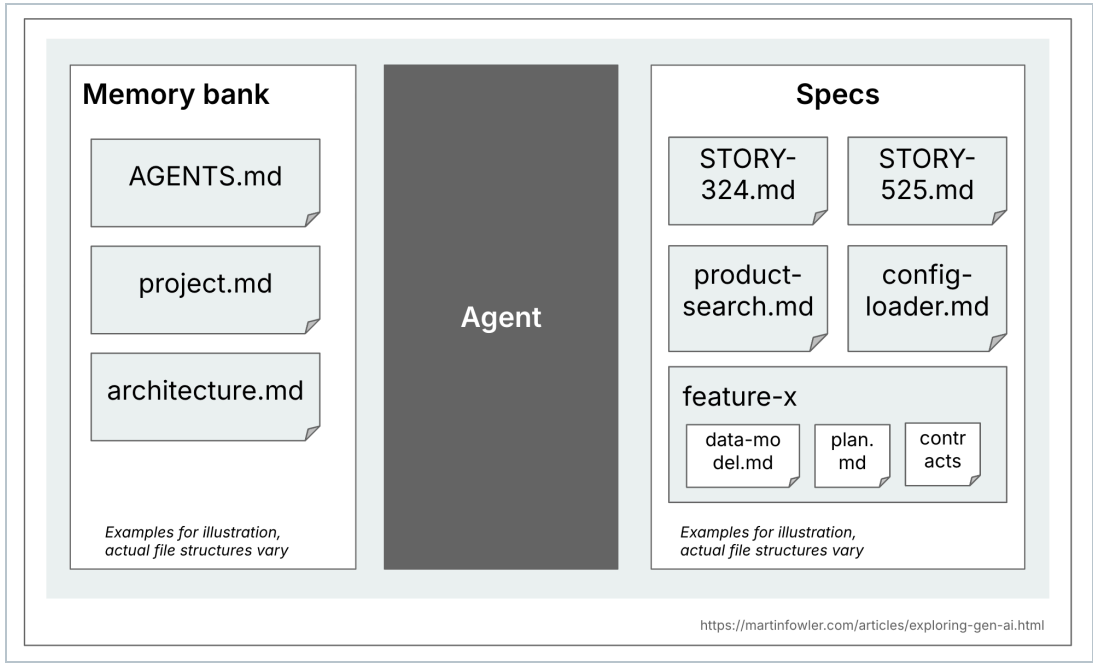
The spec is the main source file over time. Only the spec is edited by the human, who never touches the code.

Only the third row forbids a human from touching code.

© 2026 Inspire | Confidential

A spec guides one change

A spec is a structured, behavior-oriented artifact in natural language that expresses software functionality and guides AI coding agents.



Memory bank and specs sit on either side of the agent

Definition and diagram: Birgitta Böckeler, martinfowler.com

Memory bank · every session

Relevant to every AI coding session in this repository, whichever feature it touches.

Specs · one task

Relevant only to the task that creates or changes that one feature.

© 2026 Inspire | Confidential

A behavior contract

A spec is a behavior contract, not an implementation plan. If implementation can change without changing externally visible behavior, it likely does not belong in the spec.

Into the spec

Externally visible behavior that users or downstream systems depend on.

Inputs, outputs and error cases.

External constraints: security, privacy, reliability, compatibility.

Scenarios that can be tested or explicitly accepted.

Not in the spec

Internal class and function names.

The choice of library or framework.

Step-by-step implementation order.

A detailed execution plan.

© 2026 Inspire | Confidential

Which commit changes the spec

Behavior changed, or only the implementation? Read the three messages first, then the row underneath.

REFACTOR	FEATURE	FIX
`refactor: extract a method from X`	`feat: add something new`	`fix: the behavior that ... is wrong`

Score each message against the standard on this page, not against the page before it.

STAYS IN THE CODE	CHANGES THE SPEC	CHANGES THE SPEC
The outside can behave exactly as before, so the spec keeps its wording.	It adds external behavior, so the spec gains it.	The named behavior is wrong, so the spec moves to the correct one.

© 2026 Inspire | Confidential

Write the behavior first

Spec first, and only the change the outside can see.

1. Write the spec first, then hand it to the agent. That is Spec First, on the Test First side.
2. A spec follows one feature. What every session in this repository needs belongs in the memory bank.
3. Change the spec when external behavior changes. A different implementation behind the same outside leaves the spec alone.

feat	add something new	New external behavior, so the spec changes
fix	a behavior is wrong	The spec moves to the correct behavior
refactor	extract a method	Stays in the code, the spec does not move

© 2026 Inspire | Confidential

INSPIRE AI

LECTURE

Follow One Change Through OpenSpec

Align on what, why and how before the code is written



© 2026 Inspire | Confidential

A chat log is not a specification

Instructions stay in a conversation. The next session cannot read them.

Chat alone

The instruction is lost with the window.

Context drifts and the agent rebuilds its own idea of the task.

Generated code misses an edge case nobody wrote down.

Generated code breaks an architecture nobody wrote down.

A spec layer in the repository

OpenSpec is a lightweight, open-source framework for specification-driven development.

The specs are markdown files under version control, read by a person and by Claude Code, Cursor or GitHub Copilot.

What was aligned lands in Git, not in one conversation.

That layer is the difference between vibe coding and a specification-driven change: the answer is in the repository before the code is.

© 2026 Inspire | Confidential

Three choices let a spec live in a real repository

All three hold at once, or the specification does not survive contact with existing code.

Living documentation

Specs live in Git beside the code.

They accumulate into the authority on what the system does.

A static wiki page goes out of date and stays out of date.

Tool-agnostic

Works with more than thirty AI coding tools and editors.

No lock-in to one proprietary IDE or one vendor service.

Brownfield-friendly

Changes are recorded as incremental deltas.

A large, messy codebase needs no large rewrite first.

Delta sections: `ADDED`, `MODIFIED`, `REMOVED`

The three delta sections are named here and defined on the delta page.

© 2026 Inspire | Confidential

Planning can go back

A traditional process locks its stages. Every OPSX action can be repeated.

	TRADITIONAL	OPSX
Steps	PLANNING, then IMPLEMENTING, then DONE.	proposal, specs, design, tasks, implement.
Going back	The step from planning to implementation is marked as final.	Any step can be run again; a new proposal starts the next round.
What drives it	A stage gate inside the process.	A slash command, or plain language to the assistant skill.

The point is not a shorter pipeline. The lock is gone, so a wrong decision costs one more round instead of the whole plan.

© 2026 Inspire | Confidential

Five commands carry one change

Each command leaves one artifact behind, so the files tell you which step you are on.

COMMAND	WHAT IT DOES	WHAT IT LEAVES BEHIND
`/opsx:explore`	Reads the codebase, compares options, and talks the task down to one plan.	A plan you are free to discard. Nothing is committed.
`/opsx:propose`	Drafts the change before any implementation.	proposal.md, design.md, tasks.md, and a spec delta under openspec/
`/opsx:apply`	Implements the tasks already agreed.	Code that follows those tasks, not a new reading of the request.
`/opsx:verify`	Checks the code against the spec.	Evidence of where code and specification disagree.
`/opsx:archive`	Merges a finished delta back into the permanent specs.	openspec/specs/ updated, so the documentation moves with the codebase.

Only archive writes into openspec/specs/; the other four stop inside the change.

© 2026 Inspire | Confidential

specs is what the system does now; changes is this one

One change, one folder. Everything that belongs to this change sits inside it.

```
openspec/
├── specs/
│   └── <domain>/spec.md
├── changes/
│   └── <change-name>/
│       ├── proposal.md
│       ├── design.md
│       ├── tasks.md
│       └── specs/<domain>/spec.md
└── config.yaml
```

`specs/<domain>/spec.md` is the current behavior; the same path inside a change is that change's delta. `config.yaml` is optional.

© 2026 Inspire | Confidential

A spec answers what it does; a change holds this one

A change proposes edits to a spec. Archiving is the step that writes them in.

Spec	Change
Describes how one part of the system behaves.	One unit of work in openspec/changes/<name>/.
Lives in openspec/specs/, grouped by domain, built from requirements and scenarios.	Holds proposal, design, tasks, and the spec edits this change introduces.
openspec/specs/ as a whole is the source of truth for behavior already agreed.	One change is one feature or one fix.
One consistent answer to the question of what this software does.	

The arrow runs from the right column to the left, and archive is the only label it carries.

© 2026 Inspire | Confidential

A delta writes only the part that changes

No change restates the whole behavior of the system.

ADDED	MODIFIED	REMOVED
Behavior this change introduces.	Agreed behavior this change replaces.	Behavior this change takes away.

A delta sits inside the change, so an old system is edited cleanly instead of re-specified. A domain is a logical group of specs, named the way you describe the system.

DOMAIN	WHAT IT GROUPS
auth/	Login, sessions and permissions, named the way you would explain sign-in.
payments/	Pricing, invoices and refunds, named the way you would explain money.
ui/	Screens, navigation and states, named the way you would explain what people see.

© 2026 Inspire | Confidential

After archiving, the repository remembers what the system does

1. Create one change: one feature or one fix, in openspec/changes/<name>/.
2. Write the delta: ADDED, MODIFIED and REMOVED, and nothing else.
3. Run /opsx:verify and read the places where code and specification disagree.
4. Run /opsx:archive, which merges the delta back into openspec/specs/.

The next session installs this framework and walks one requirement to its first archive. This is the file the round will write into.

© 2026 Inspire | Confidential

BREAK

Take a short break



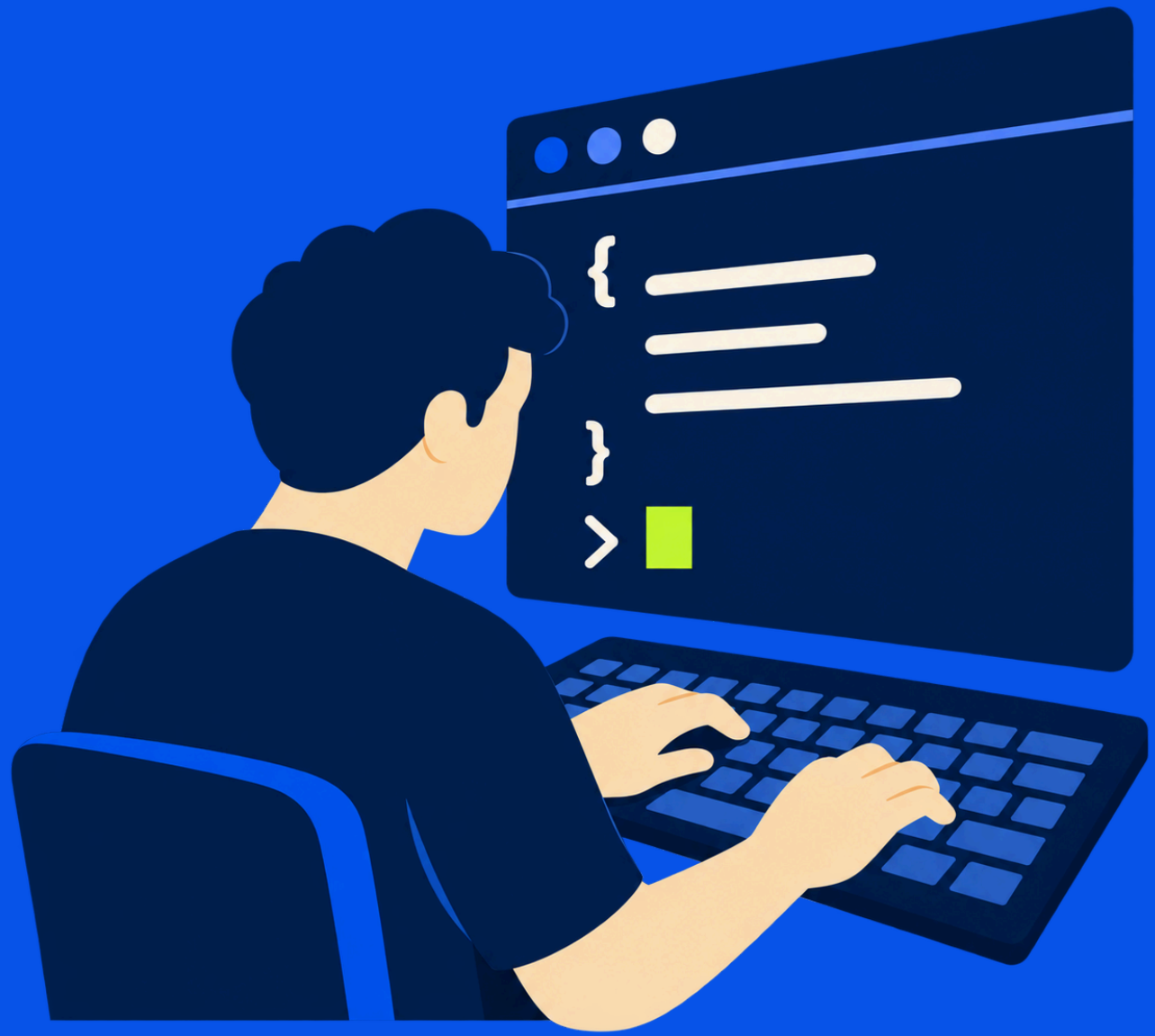
© 2026 Inspire | Confidential

INSPIRE AI

DEMO

Install OpenSpec and Reach the First Archive

One empty directory, one entry, one archive



© 2026 Inspire | Confidential

Check Node first, then install the command globally

This session stops at the first archive.

```
node --version
```

22.12.0 or higher. A lower version does not install what comes next.

```
npm install -g @fission-ai/openspec@latest
```

© 2026 Inspire | Confidential

Create the directory and open it in Qoder

```
mkdir sdd-onboard
```

```
cd sdd-onboard
```

```
openspec init --tools qoder
```

qoder is the init target	init writes the workflow commands into that tool. Any other name and the next step has nothing to call.
Open it as a workspace	Creating the folder is not enough. The commands exist inside the workspace, so the folder has to be opened as one.

© 2026 Inspire | Confidential

In the profile, make two choices only

```
openspec config profile
```

Choose workflows only	This is the choice that decides which set of commands appears in the next step.
Check every option	Everything this profile offers is switched on for the lab. Do not pick a subset here.

The option names are not listed on this page. Follow the menu on your own screen, and copy nobody else's screen.

© 2026 Inspire | Confidential

The quick path stays in core; today takes the expanded path

PATH	WHAT RUNS
core quick path	/opsx:explore (optional) → /opsx:propose → /opsx:apply → /opsx:sync → /opsx:archive
expanded path	/opsx:new → /opsx:ff or /opsx:continue → /opsx:apply → /opsx:verify → /opsx:archive

> `@openspec-new-change` is the only thing you type. The path underneath is its business, not yours.

Both paths live in the same profile. The class uses the expanded one because every stage leaves something to look at.

© 2026 Inspire | Confidential

Pick no stack yet; settle the first behavior first

Send this sentence first

Help me explore a task tracker from scratch. I have not picked a stack, only frontend with local storage. Compare the options and help me choose the first behavior to build.

Then state the requirement

Requirement as typed in class: A kanban board that tracks tasks

The first sentence asks for options, not code. The second sentence is the first behavior, and it is what enters the change.

© 2026 Inspire | Confidential

One entry, all the way to the first archive

```
new · ff or continue · apply · verify · archive
```

Keep typing `@openspec-new-change` and stop at the first archive.

openspec/specs/ now holds the first behavior of the board. That folder is the reason this session exists.

Name the second change before you stop, but do not start it. Three more requirements wait in the next session.

© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Run Three More Requirements Through One Entry



© 2026 Inspire | Confidential

For the next requirements it is still this one entry

One entry only	The expanded path runs underneath @openspec-new-change. Do not switch to the quick path halfway through.
One requirement, one change	These are three rounds, not three items inside one change.
Read the directory first	At every stage the directory answers whether that stage passed. Nothing else does.

© 2026 Inspire | Confidential

Every stage is checked against the directory

STAGE	WHAT HAS TO BE THERE
new, ff or continue	openspec/changes/<name>/ holds proposal.md, design.md, tasks.md and the delta inside it
apply	The tasks land in the code
verify	The code matches that delta
archive	The delta merges into openspec/specs/

If the directory did not change, the stage has not passed. A convincing summary is not evidence.

© 2026 Inspire | Confidential

One requirement, one round of change

1 · Drag, edit, archive	Three behaviors on one card. Write them into this round and archive the round.
2 · Keep drag consistent	The same movement after the change. This one is small, so it tests the process, not the feature.
3 · Edit and comment	Changed content and a posted comment, in its own round.

Finish one round before you open the next. Merging two requirements into one change does not count.

© 2026 Inspire | Confidential

The archive in requirement one is not the archive command

Card archive · a behavior

Drag, edit, archive and comment are behaviors the board has to show.

They are written into the delta like any other behavior, and the round proves them.

Command archive · a merge

archive merges this delta back into the Living Spec.

It is the only command that writes into openspec/specs/.

Both words appear inside the first requirement. Separate them before the delta is written, not after.

© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

INSPIRE AI

SHOWCASE

Explain the Change, the Delta and the Living Spec



© 2026 Inspire | Confidential

Three words, three places

WORD	WHERE IT LIVES
Change	openspec/changes/<name>/ — the work package for this round
Delta spec	specs/<domain>/spec.md inside the change — only ADDED, MODIFIED and REMOVED
Living Spec	openspec/specs/<domain>/spec.md — the current state the team agreed on

Finish the change that never reached archive before you present anything.

© 2026 Inspire | Confidential

Read the change before archive, read specs after

Before archive	After archive
The behavior lives in the delta.	The delta merged into openspec/specs/.
The Living Spec has no such paragraph yet, or still holds the previous round.	This paragraph is now the current state.

To change this behavior again, the next delta uses **MODIFIED** and edits that current state.

archive is the only arrow between the two columns.

© 2026 Inspire | Confidential

Say which layer this round changed

1. What is inside this change folder?
2. Which markers did the delta use: ADDED, MODIFIED or REMOVED?
3. After the archive, which paragraph of openspec/specs/ moved?

The third question is the one that stays after the day ends. Answer it with openspec/specs/ open.

© 2026 Inspire | Confidential

The next requirement still starts at @openspec-new-change

@openspec-new-change

Every next requirement enters here.

archive

Close the round and merge the delta.

openspec/specs/

Check that the behavior is remembered.

No thank-you page, no question page. The round closes when the Living Spec holds the behavior.

© 2026 Inspire | Confidential

LECTURE

Explain the Homework and Tomorrow's Entry



© 2026 Inspire | Confidential